

Интеграция Tensor Flow Lite Micro на MIK32 Amur на базе ELBEAR ACE-UNO и попытка в microspeech.



TensorFlow Lite

Клянцевич Илья, Университет Иннополис, 2026

Что за Tensor Flow Lite Micro (TFLM).

[Tensor Flow Lite Micro \(TFLM\)](#) - фреймворк для запуска моделей машинного обучения на микроконтроллерах. (AKA LiteRT for Microcontrollers)

Обычно требуется что-то помощнее M1K32. Google рекомендует использовать платы из [этого](#) списка.

Посмотрев на эти примеры я не нашел платы с менее 64 МГц процессором и 144 KB SRAM.

The following development boards are supported:

- [Arduino Nano 33 BLE Sense](#)
- [SparkFun Edge](#)
- [STM32F746 Discovery kit](#)
- [Adafruit EdgeBadge](#)
- [Adafruit LiteRT for Microcontrollers Kit](#)
- [Adafruit Circuit Playground Bluefruit](#)
- [Espressif ESP32-DevKitC](#)
- [Espressif ESP-EYE](#)
- [Wio Terminal: ATSAM51](#)
- [Himax WE-I Plus EVB Endpoint AI Development Board](#)
- [Synopsys DesignWare ARC EM Software Development Platform](#)
- [Sony Spresense](#)

А что если все таки попробовать?

Начнем с билда - надо разобраться с тем как прилинковать TFLM в наш существующий проект.

Клонирова

И,
third_party/tflite
lite/micro/toc

По
riscv32_gen
создать .inc
МК и

mik32_

```
op_resolver = new (kOpStorage) tflite::MicroMutableOpResolver<4>();
if (op_resolver->AddReshape() != kTfLiteOk) {
    return (last_error = -24);
}
if (op_resolver->AddFullyConnected() != kTfLiteOk) {
    return (last_error = -22);
}
if (op_resolver->AddDepthwiseConv2D() != kTfLiteOk) {
    return (last_error = -21);
}
if (op_resolver->AddSoftmax() != kTfLiteOk) {
    return (last_error = -23);
}

interpreter = new (kInterpStorage) tflite::MicroInterpreter(model, *op_resolver, shot_pool, kArenaBytes);

const TfLiteStatus alloc_st = interpreter->AllocateTensors();
if (alloc_st != kTfLiteOk) {
    last_error = -14;
    xprintf("[TFLM] alloc fail %d used=%u\r\n", (int)alloc_st,
            (unsigned)interpreter->arena_used_bytes());
    return last_error;
}

if (cache_tensor_ptrs() != 0) {
    last_error = -15;
    return last_error;
}

initialized = true;
return 0;
```

party/tflite-micro:
-f
ls/make/Makefile \
mik32
with_logs microlite

ет здесь:
cro/gen/mik32
with_logs_gcc

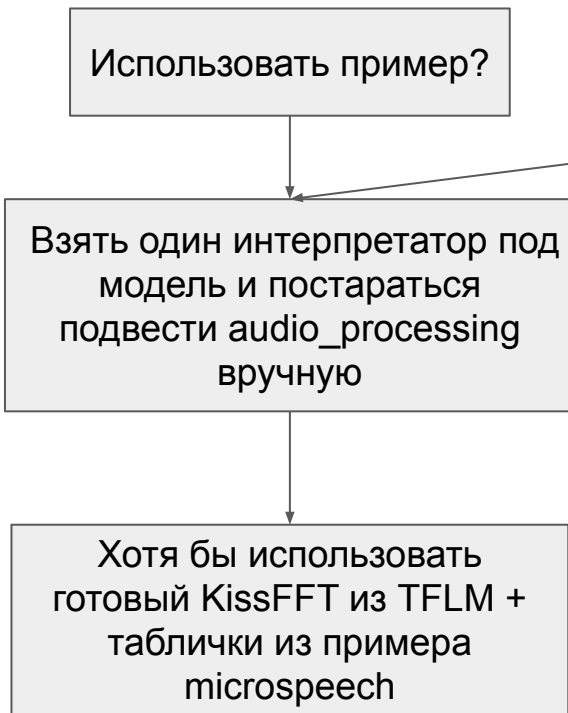
CMake

слинковать

Ура у нас есть билд! Теперь спланируем работу.

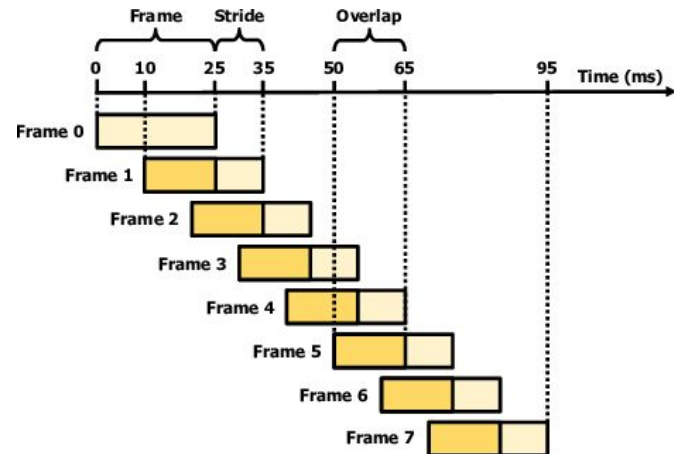
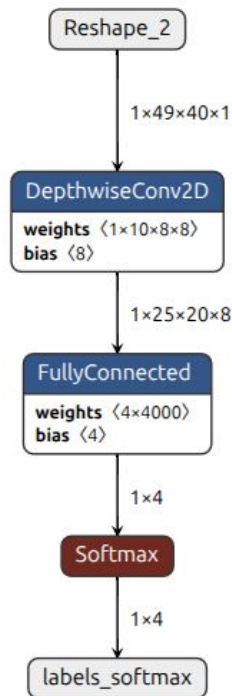
Я решил взять заведомо невозможную задачу: microspeech модуль.

```
third_party/tflite-micro/tensorflow/lite/micro/
examples/micro_speech/micro_speech_test.cc:
constexpr size_t kArenaSize = 28584; //
shared tensor arena
```

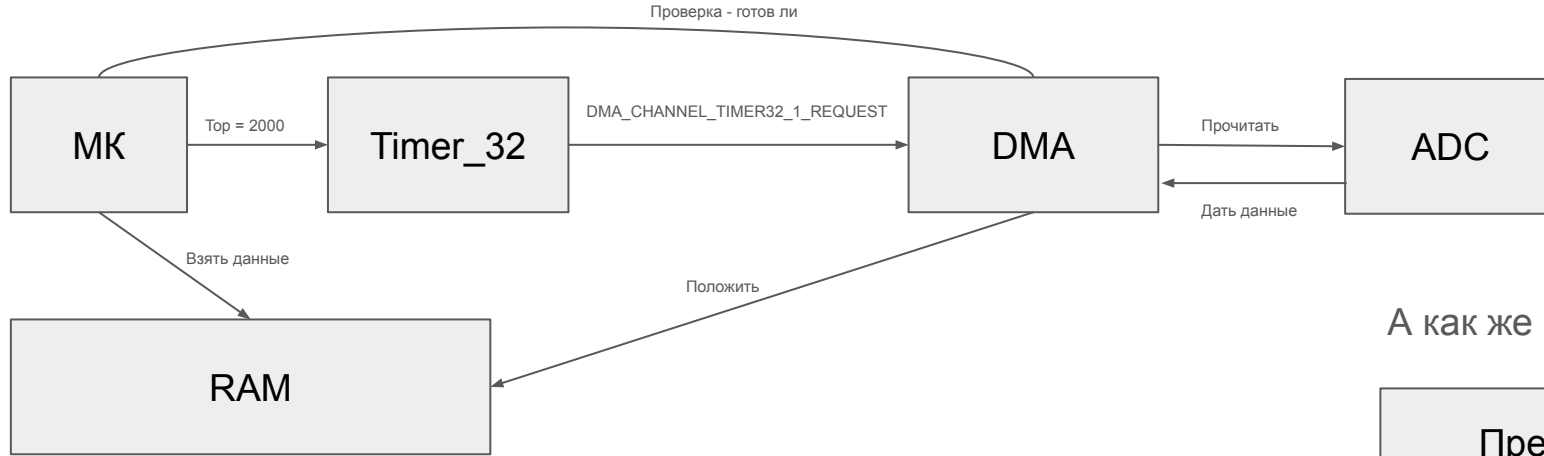


Немного о модели.

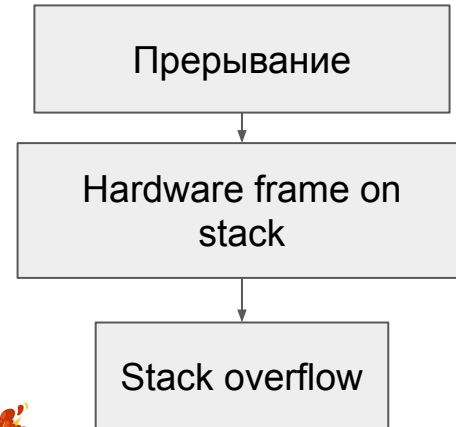
- 30мс окно;
- 20мс шаг окна;
- 16 кГц частота сбора данных;
- Данные из ADC должны быть центрированы под int16_t для правильной обработки аудио;
- Одноканальная;



А что по hardware?



А как же прерывания!?



Первые проблемы...

Не хватает скорости
чтобы сэкономить на
месте и
обрабатывать аудио
маленькими
буферами



Долго и мучительно
разбираться с
audio_processing и
попытками
оптимизировать FFT

Требуемая частота
получения и
обработки аудио - 16
кГц. В моем случае
обработка замедляла
процесс до ~1кГц

Не хватает памяти
чтобы сохранить
всё аудио и затем
обработать все
одним заходом



А что если
переиспользовать то что мы
можем переиспользовать?

TFLM буффер

FFT буффер

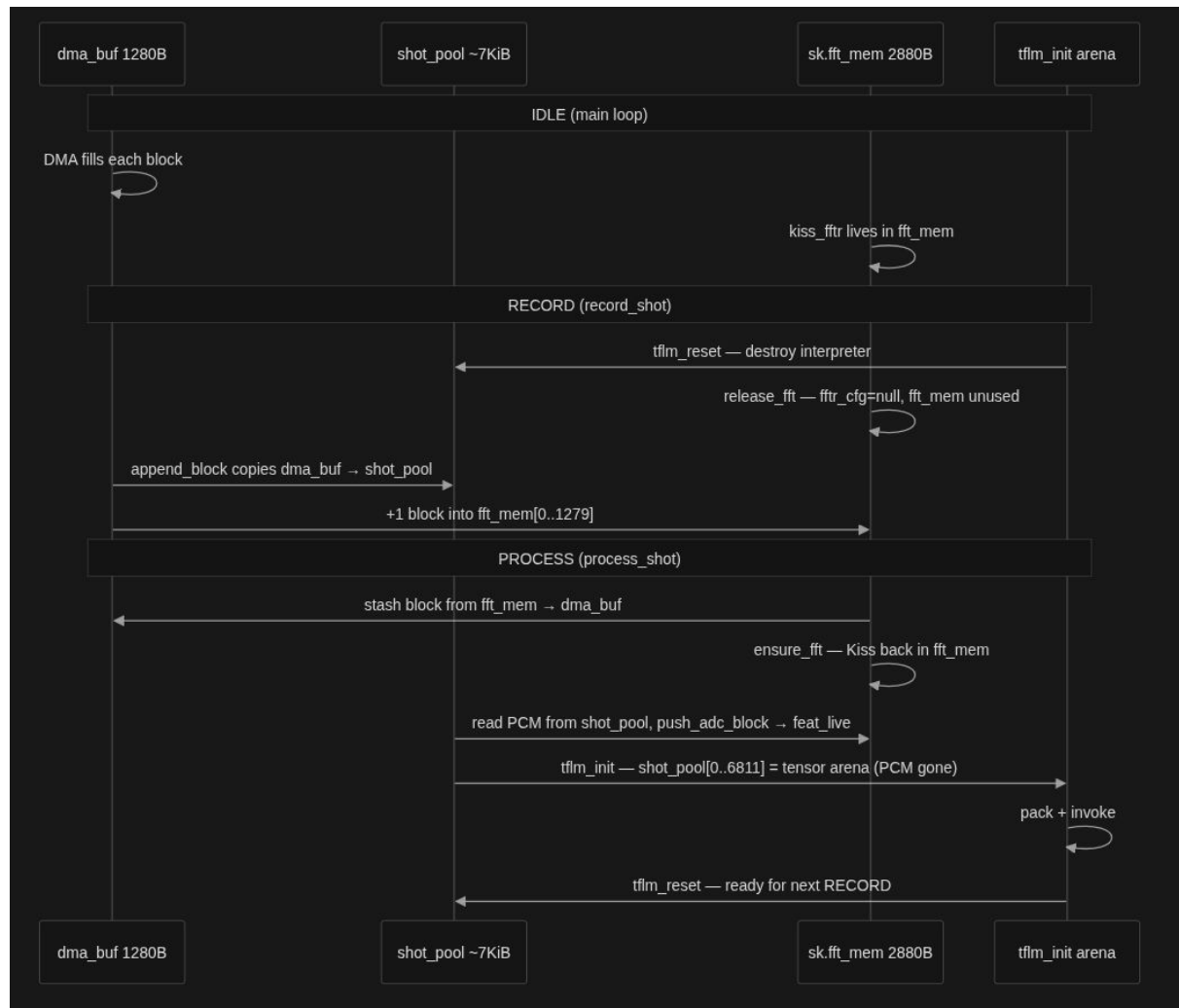
Про память.

Есть три состояния: IDLE, RECORD, PROCESS.

IDLE - читаем информацию с ADC на 16 КГц и проверяем есть ли на ней скачки.

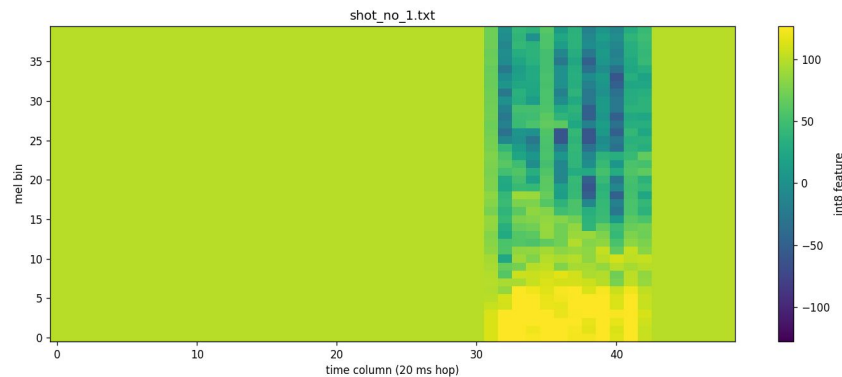
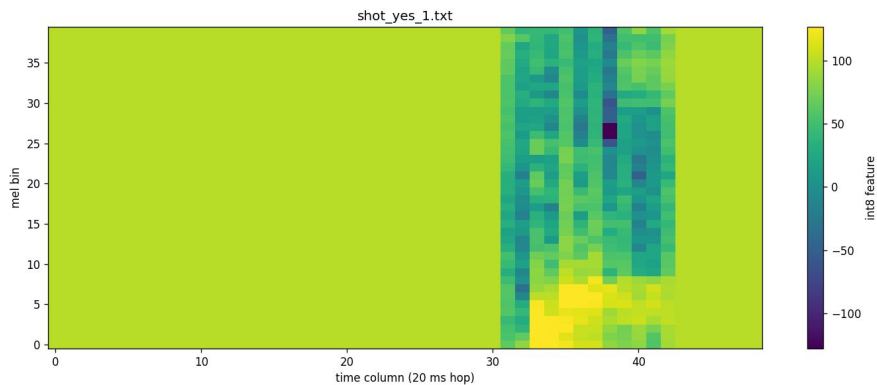
RECORD - появился скачок -> начинаем слушать. Используем буффер от модели и один блок кладем сразу в FFT.

PROCESS - из буффера записи передаем значения на обработку FFT. Как обработка закончена отдаем память модели.



Что модель получала на вход?

Так как модель училась на 1 секундных отрывках речи, а скорость сбора данных 16 кГц с 16 битным ADC - нужен буффер размером в 32KB. К сожалению никакие трюки не превратят 16KB в 32KB. Что означает - заполнить голосом я мог где то 200мс от 1с (с ~7KB под запись).



Что в конце концов с моделью?

Несмотря на все усилия модель так как не была обучена на подобных данных и точно не готова ко всем прелестям работы на платформе которая в 2-4 раза менее производительна, чем **минимальные** требования - модель чаще всего отвечает либо одинаковыми весами на всех классах (означает что она не понимает что происходит) или сочетаниями одинаковых весов на “yes”, “no”, “unknown”.

```
001000 [BOOT] taking silence blocks, 0/20
[BOOT] taking silence blocks, 1/20
[BOOT] taking silence blocks, 2/20
[BOOT] taking silence blocks, 3/20
[BOOT] taking silence blocks, 4/20
[BOOT] taking silence blocks, 5/20
[BOOT] taking silence blocks, 6/20
[BOOT] taking silence blocks, 7/20
[BOOT] taking silence blocks, 8/20
[BOOT] taking silence blocks, 9/20
[BOOT] taking silence blocks, 10/20
[BOOT] taking silence blocks, 11/20
[BOOT] taking silence blocks, 12/20
[BOOT] taking silence blocks, 13/20
[BOOT] taking silence blocks, 14/20
[BOOT] taking silence blocks, 15/20
[BOOT] taking silence blocks, 16/20
[BOOT] taking silence blocks, 17/20
[BOOT] taking silence blocks, 18/20
[BOOT] taking silence blocks, 19/20
[BOOT] mid=1660 gain=8 pk=1000 cap=7180B
[SHOOT] start to capture voice, pk = 1155
[SHOT] block 1 read
[SHOT] block 2 read
[SHOT] block 3 read
[SHOT] block 4 read
[SHOT] block 5 read
[SHOT] blk=6 hops=10 pk=1155 cls=no
[SHOOT] start to capture voice, pk = 1015
```

```
[SHOT] hops=12 pk=927 col33..44 cls=yes fft=3042ms init=265ms inv=19317ms
[TFLM] int8: -128 -48 -40 -40 q8: sil0 unk312 yes343 no343 -> yes
```

Что выучил?

Несмотря на очевидный провал с точностью модели у меня вышло:

- Собрать полноценную систему между TIM<->DMA<->ADC, работающую без участия МК;
- Как следует познакомиться с TFLM. Важный опыт для дальнейших проектов и экспериментов с TinyML;
- Поиграться с памятью ценой тысяч попыток (общий буфер, получилось что-то типа контролируемого heap).



Спасибо за внимание!

